

RecruitAIr

MLOPS project milestones 4-6

[Criteria extractor repository](#)

[Criteria evaluator repository](#)

[Dispatcher repository](#)

[Deployment repository](#)

Made by:

- Alfonso Brown
- Tania González
- Virginia Nicosia
- Marc Parcerisa
- Daniel Reverter

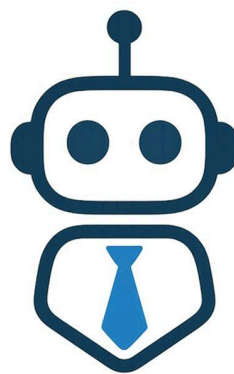


Table of contents

Milestone 4: Model deployment – API	3
ML-based component/system architecture	4
Architecture	4
Design patterns	5
Design decisions	5
Model deployment	6
Deployment technologies	6
API design	8
Dispatcher API	8
Criteria Extractor API	8
Criteria Evaluator API	9
API testing	9
Criteria Extractor API Testing	10
Criteria Evaluator API Testing	10
Dispatcher API Testing	10
Milestone 5: Model deployment – Model packaging	11
Container and Orchestration	11
CI/CD	12
Milestone 6: Monitoring	12
Resource monitoring	12
Model performance	14
Cycle: Feedback loops and retraining	14
Self-evaluation of the project	14

Milestone 4: Model deployment – API

The architecture of RecruitAIr was designed to remain modular, explainable and easily maintainable as the project grows. Following the same principles applied in the earlier stages of the project, we structured the system around a clear L-based architecture, distinguishing between the presentation layer of the application and the two independent AI subsystems. This design not only ensures full transparency and separation of responsibilities, but also enables each component to evolve independently while remaining fully compatible with the rest of the ecosystem.

ML-based component/system architecture

Architecture

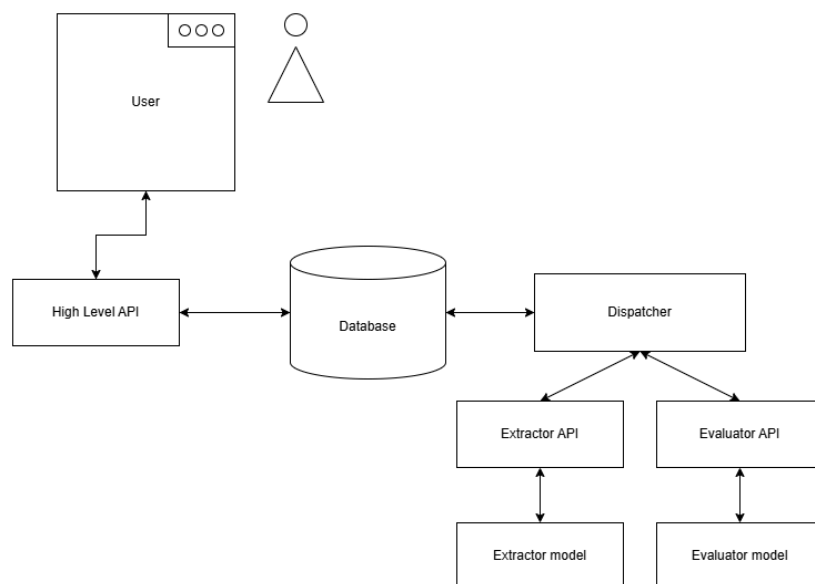


Figure 1: System component architecture overview

The system consists of four main components, each serving a distinct purpose in the recruitment workflow. The Criteria Extractor API processes job offer descriptions to extract structured evaluation criteria; it uses FastAPI with LLM-based extraction powered by Dolphin3 via Ollama, taking raw job offer text as input and producing JSON containing structured criteria with descriptions and importance weights ranging from 0 to 1. This service is deployed as a standalone API accessible via HTTP.

The Criteria Evaluator API evaluates individual applicant CVs against specific job criteria. Built with FastAPI and a fine-tuned Qwen3-0.6B model loaded from MLflow, it takes an applicant CV text and a single criterion description as input, returning a suitability score between 0 and 1. The model is loaded from the MLflow registry using a remote tracking URI; after resolving initial artifact storage challenges, the API successfully loads models from the MLflow tracking server in production environments, with remote artifact storage properly configured.

The Dispatcher Service orchestrates the entire workflow, managing job offers, applicants, criteria, and scores. It consists of a REST API built with FastAPI for managing all entities, along with two background workers: an Extractor Worker that polls for pending job offers and calls the Criteria Extractor API, and an Evaluator Worker that processes applicant-criterion pairs and calls the Criteria Evaluator API. The service uses PostgreSQL as its database for persistent storage, with SQLAlchemy (async) for database access and Alembic for migrations. The entire service is deployed as multiple containerized services including the API, workers, database, and migration jobs.

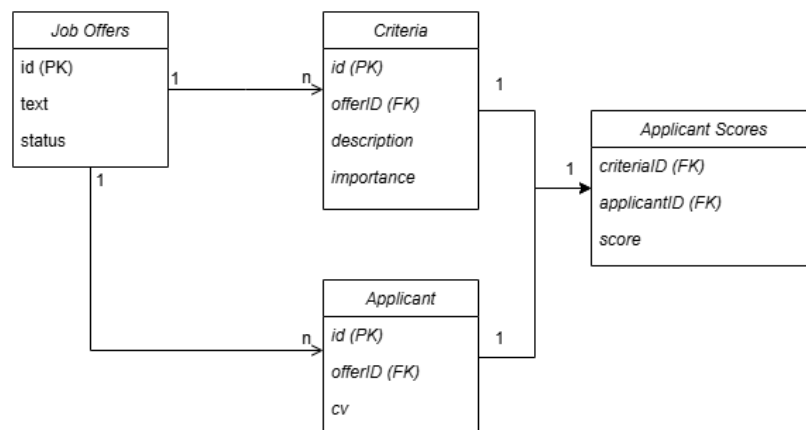


Figure 2: Job offers database schema

Finally, the Deployment Infrastructure repository contains Helm charts and Kubernetes manifests for deploying the entire stack. Built using Helm 3 and Kubernetes, with Kind for local testing, it provides configurable resource limits and autoscaling, health checks and readiness probes, ingress configuration for external access, and database persistence with PostgreSQL.

Design patterns

The system employs several well-established design patterns to achieve its goals. The microservices pattern ensures that each ML component (Extractor and Evaluator) is a separate service that can be scaled independently, allowing for resource optimization and fault isolation. The worker pattern handles asynchronous processing for criteria extraction and applicant evaluation, preventing blocking operations in the main API and enabling efficient batch processing.

The API Gateway pattern is implemented through the Dispatcher API, which acts as the main entry point coordinating between services; this centralizes routing, authentication and request handling while keeping the ML services focused on their core inference tasks. The repository pattern abstracts database access through SQLAlchemy models and sessions, providing a clean separation between business logic and data persistence. Finally, FastAPI's dependency injection system manages database sessions and model loading, ensuring proper resource lifecycle management and testability.

Design decisions

The cloud provider selection reflects our commitment to portability and flexibility. While MLflow tracking is hosted on [AWS ECS](#) with remote artifact storage configured, enabling the APIs to load models and prompts from any location, the application itself can run on any Kubernetes-compatible platform; this portability is achieved through Helm charts that abstract cloud-specific configurations, container images stored in GitHub Container Registry (GHCR) for universal access, standard Kubernetes resources that work across providers, and remote MLflow artifact storage enabling distributed model loading.

The separation of ML components into distinct services was driven by several considerations: the Criteria Extractor and Evaluator have different resource requirements, with the Extractor using LLMs and the Evaluator using fine-tuned models, necessitating different scaling strategies. They can be scaled independently based on workload, allowing for cost optimization. Additionally, they can be developed and deployed by different teams, facilitating parallel development, and failure in one component doesn't cascade to the other, improving system resilience.

Asynchronous processing through background workers was chosen to prevent blocking the main API during long-running ML inference operations. This approach enables batch processing for efficiency, allows retry logic and error handling without affecting user requests, and supports horizontal scaling of workers based on queue depth. The database-first architecture uses PostgreSQL as the central state store, enabling transactional consistency for job offers, applicants, criteria, and scores, query capabilities for filtering, pagination, and reporting, audit trails through timestamps and status tracking, and worker coordination through database queries and advisory locks.

Model deployment

The ML models are deployed as RESTful API services using a microservices architecture, enabling independent scaling, versioning, and maintenance of each ML component. This approach allows each model to be updated, scaled, or replaced without affecting other components, providing flexibility in managing the ML infrastructure.

Deployment technologies

For the Criteria Extractor Model, the deployment method uses an API service with runtime model loading. The model source is the MLflow Prompt Registry, with Dolphin3 via Ollama serving as the LLM backend, which can be either local or remote. The loading strategy involves loading prompt templates at application startup via the MLflow GenAI API, initializing the LLM client on startup for faster inference, and using FastAPI's lifespan context manager for preloading. The technology stack includes langchain-ollama for LLM integration, mlflow-genai for prompt versioning and management, and structured output parsing with JSON schema validation.

The Criteria Evaluator Model follows a similar deployment approach but uses the MLflow Model Registry as its source. It employs a fine-tuned PyTorch model (Qwen3-0.6B with a custom regression head) that is loaded at application startup via `mlflow.pytorch.load_model()`. Model artifacts including weights and tokenizer are

downloaded from the remote MLflow tracking server, and the model is moved to the appropriate device (CPU/GPU) based on availability. The technology stack includes mlflow for model versioning and artifact management, PyTorch for model inference, a custom tokenizer (ResumeAndCriteriaTokenizer) loaded from model artifacts, and remote artifact storage enabling distributed model loading.

The containerization approach includes multi-stage builds for optimized image sizes, non-privileged user execution for security, dependency caching for faster rebuilds, and environment variable configuration for model/prompt versions.

Local development uses Docker Compose to orchestrate all services locally, including MockServer for testing without real ML APIs and a PostgreSQL container with persistent volumes. In production, Kubernetes serves as the container orchestration platform, with Helm Charts providing package management for Kubernetes deployments. Kubernetes Services enable internal communication, an AWS ALB Ingress Controller handles external access, and Horizontal Pod Autoscaler manages scaling based on CPU/memory metrics.

The MLflow integration provides a comprehensive model registry and versioning system. The tracking server is hosted on [AWS ECS](#), with remote storage configured (S3 or similar) enabling distributed access. The system includes a centralized model registry for versioning and promotion workflows, and a prompt registry for versioned prompt templates used in LLM-based extraction. This setup provides model versioning and lineage tracking, A/B testing capabilities, rollback to previous model versions, and environment-specific model promotion from staging to production.

The deployment workflow follows a structured process: first, models are trained and logged to the MLflow tracking server, while prompts are tested and versioned in the MLflow prompt registry, with performance metrics evaluated against benchmark datasets. When performance criteria are met, models and prompts are promoted to the production registry with version tags assigned (models:/criteria-evaluation/2). For API deployment, Docker images are built and pushed to GitHub Container Registry (GHCR), Kubernetes deployments are updated with new image tags, environment variables are configured with MLflow tracking URI and model/prompt versions, and health checks verify successful model loading. At runtime, API containers start and execute lifespan startup logic, models and prompts are downloaded from the MLflow registry, models are loaded into memory (GPU if available) and the API becomes ready to serve requests.

Infrastructure provisioning and deployment orchestration are unified using the AWS Cloud Development Kit (CDK) in Python, establishing a Infrastructure as Code (IaC) foundation. The CDK stack automates the creation of a dedicated VPC and an Amazon EKS cluster, configuring managed node groups with Amazon Linux 2023 to ensure OS compatibility and security compliance. To support stateful workloads and external access, the stack installs essential cluster add-ons, including the Amazon EBS CSI driver for persistent database storage and the AWS Load Balancer Controller, which provides native integration with AWS Application Load Balancers. This setup wraps the application's Helm charts within the CDK logic, enabling the deployment of the full microservices architecture (Dispatcher, Extractor, and Evaluator) while abstracting the complexity of manual manifest management.

Key deployment features include lazy loading prevention through model preloading at startup to avoid first-request latency, health checks via Kubernetes probes that verify model availability, graceful degradation where APIs can fall back to dummy models if MLflow is unavailable during development, configurable resource management with CPU/memory limits per service, horizontal scalability of API pods based on load, and observability through model version inclusion in API responses for traceability.

API design

Dispatcher API

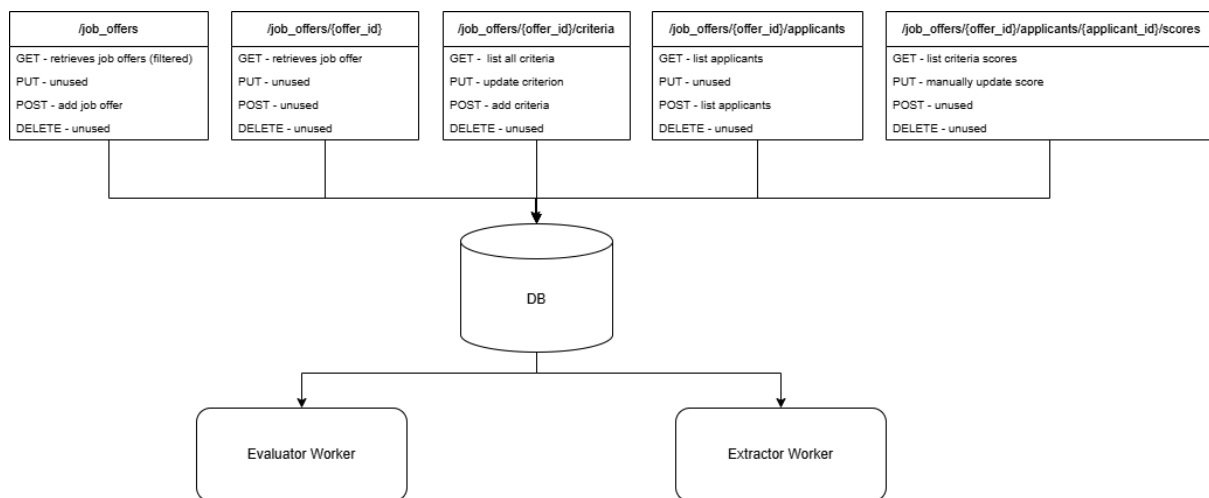


Figure 3: Dispatcher API endpoints

The Dispatcher API provides comprehensive endpoints for managing the recruitment workflow, with a base URL of `/api` (configurable via `RECRUITAIR_API_ROOT_PATH`). The API is organized into several resource groups:

- **Job Offers** (`/job_offers`): The API allows creating new job offers via POST, listing job offers with filtering (status, text search) and pagination via GET, and retrieving details of specific job offers via GET with the offer ID.
- **Criteria** (`/job_offers/{offer_id}/criteria`): For each job offer, the API provides endpoints to get all criteria via GET, add criteria via POST (which can be called by the extractor worker or manually), and update a criterion's description or importance via PUT.
- **Applicants** (`/job_offers/{offer_id}/applicants`): The API supports creating applicants for a job offer via POST with batch creation support, and listing applicants with filtering and pagination via GET.
- **Scores** (`/job_offers/{offer_id}/applicants/{applicant_id}/scores`): The API provides endpoints to get all scores for an applicant via GET, get a score for a specific criterion via GET with criterion ID, and update a score for manual override via PUT.
- **Health** (`/health`): A standard health check endpoint returns service status.

The API design features include automatic OpenAPI documentation generation with Swagger UI available at `/docs`, tagged endpoints organized by resource type (Job Offers, Criteria, Applicants, Scores, Health), cursor-based pagination with limit and offset parameters, query parameters for text search and status filtering, comprehensive Pydantic

validation for all inputs, and consistent error responses with appropriate HTTP status codes (404, 422, etc.).

Criteria Extractor API

The Criteria Extractor API is implemented using FastAPI and extracts structured criteria from job offer descriptions. The base URL is / (or /extractor when deployed behind ingress), providing a simple and intuitive interface for criteria extraction.

The primary endpoint is POST /eval, which extracts key criteria from a job offer description: it accepts a request body containing the offer_text field, a string between 1 and 10,000 characters and the response returns a JSON object with a criteria array, where each criterion includes a description and an importance score between 0 and 1. The API uses the MLflow prompt registry for prompt management, with Dolphin3 via Ollama as the LLM backend, configurable via environment variables. This prompt versioning enables A/B testing and gradual rollouts. Error responses include 422 for validation errors (invalid input format or length) and 500 for model extraction failures.

The health check endpoint GET /health returns a simple JSON object with {"status": "ok"}, used by Kubernetes probes to verify service availability.

The API design follows RESTful principles using standard HTTP methods and status codes. Type safety is ensured through Pydantic models that validate all requests and responses. The MLflow prompt registry enables version control and experimentation, while structured error responses provide appropriate HTTP status codes. CORS is enabled for cross-origin requests (configurable), and lifespan management preloads prompts at startup for faster inference.

Criteria Evaluator API

The Criteria Evaluator API is implemented using FastAPI and evaluates applicant CVs against specific criteria. The base URL is / (or /evaluator when deployed behind ingress).

The main endpoint is POST /eval, which evaluates an applicant CV against a single criterion: it accepts a request body with criteria_description (1-2000 characters) and applicant_cv (1-20000 characters) and the response includes a score between 0 and 1, the model_version from MLflow (e.g., models:/criteria-evaluation/2) and elapsed_seconds for latency tracking. The API uses the MLflow model registry (models:/criteria-evaluation/{version}) with a fine-tuned Qwen3-0.6B model loaded at startup; model versioning enables traceability and rollback capabilities. Error responses include 422 for validation errors and 500 for model prediction failures.

The health check endpoint GET /health follows the same pattern as the Extractor API, returning {"status": "ok"} for Kubernetes probes.

The API design principles mirror those of the Extractor API: RESTful architecture, type safety through Pydantic, observability through model version and latency metrics in

responses, structured error handling, CORS support, and lifespan management for model preloading.

API testing

Both APIs include comprehensive unit and integration tests using pytest and FastAPI's TestClient, ensuring reliability and correctness of the API implementations.

Criteria Extractor API Testing

The test coverage includes successful extraction scenarios where valid job offers return structured criteria with descriptions and importance scores, input validation tests that verify empty or invalid inputs return appropriate 422 errors, health endpoint tests that confirm /health returns the expected response, and mock model usage to avoid loading the real Ollama LLM during tests, ensuring fast and reliable test execution.

An example test verifies that a job offer containing Python and ML keywords returns appropriate criteria with importance scores between 0 and 1, and includes proper description fields. This approach allows developers to run tests quickly without requiring the full LLM

Criteria Evaluator API Testing

The tests cover successful evaluation scenarios where valid requests return scores in the [0, 1] range with correct model version information, input validation that ensures empty or invalid inputs return appropriate 422 errors, health endpoint verification, and mock model usage to avoid loading the real MLflow model during tests, ensuring fast and reliable test execution.

Example tests verify that evaluation requests return properly formatted responses with scores, model versions, and elapsed time measurements, all within expected ranges and formats.

Dispatcher API Testing

The Dispatcher API test suite includes comprehensive coverage across all endpoint groups:

- **Job offer** endpoint tests cover creating job offers with valid and invalid data, listing job offers with pagination and filtering, retrieving specific job offers by ID, and status filtering (PENDING, DONE).
- **Criteria** endpoint tests verify creating criteria for job offers, retrieving criteria lists, updating criterion descriptions and importance, and error handling for invalid job offers.
- **Score** endpoint tests cover retrieving scores for applicants, updating scores manually, and error handling for non-existent applicants or criteria.

The test infrastructure uses pytest fixtures for database setup and teardown, SQLite in-memory database for fast test execution, FastAPI's TestClient for HTTP endpoint testing, and comprehensive assertions for response structure and data validation.

CI/CD integration ensures tests run automatically on push to main branch, pull requests (with coverage reporting), and tag creation (before building Docker images). Coverage

reporting includes coverage percentage badges and links to detailed HTML coverage reports posted as PR comments, providing visibility into test quality and code coverage.

Milestone 5: Model deployment – Model packaging

Container and Orchestration

Containerization is central to the architecture of RecruitAIr, ensuring reproducible environments, isolated dependencies, and the ability to scale each subsystem independently. All core components (Criteria Extractor, Criteria Evaluator, and Dispatcher) are packaged as Docker images. Docker Compose is used for local development, while Kubernetes (via Helm charts) handles production deployments.

For the Criteria Extractor, the container structure separates the LLM runtime from the API logic. The Ollama service runs as its own container and stores model weights in a persistent volume, ensuring that the Dolphin3 model is downloaded only once and is immediately available across restarts. A dedicated initialization container pulls the required model on startup, eliminating cold-start latency and guaranteeing that the Extractor API only launches once the LLM backend is fully ready. The API container itself remains stateless and receives all configuration (model name, version, prompt version, MLflow URI) through environment variables, which makes it portable and allows prompt/model versioning to be controlled externally.

The Criteria Evaluator relies on a more complex Dockerfile because it must host a PyTorch model loaded from MLflow. It uses the official PyTorch CUDA runtime image to guarantee compatibility with GPU acceleration when available and to prevent dependency mismatches with the MLflow model artifacts. The container is built with a non-privileged user for security, and benefits from Docker's caching layers to reduce build times. The model is preloaded during the API startup sequence, which avoids first-request delays and integrates seamlessly with Kubernetes readiness probes.

The Dispatcher introduces additional services: a PostgreSQL database, a migration job, two workers, and the main API. The Postgres container uses persistent storage to maintain state, while health checks ensure that migrations and workers only start once the database is reachable. Database migrations run as a one-shot container to guarantee schema consistency without mixing application logic and migration logic. The two worker containers (Extractor Worker and Evaluator Worker) are built from distinct Dockerfile targets, allowing them to remain lightweight and focused on background processing. The Dispatcher API container mounts the project directory in development to support live reloading, while in production the immutable image ensures stability.

For orchestration, Docker Compose provides a complete local environment that mirrors the production architecture, including a mock API and Postgres, enabling developers to test the full pipeline without external dependencies. In production, Kubernetes provides horizontal scaling for the Extractor and Evaluator independently, health checks tied to model readiness,

persistent volumes for the LLM backend and database, and controlled rollouts through Helm charts.

Through this containerization strategy, RecruitAIr achieves consistent deployments, clean separation between components, efficient scaling, and a development workflow that closely reflects production behavior.

CI/CD

To automate the deployment pipeline, we implemented a GitHub Actions workflow that acts as the orchestrator for the AWS Cloud Development Kit (CDK). This workflow is triggered on pushes to the main branch, ensuring that every code change is automatically reflected in the production environment. The pipeline first authenticates with AWS using access keys for a specific role that only has deployment permissions. Once authenticated, the runner sets up the necessary runtime environment by installing Python, the AWS CDK CLI, and Helm. This preparation phase ensures that the runner mirrors the local development environment, preventing "works on my machine" discrepancies during deployment.

The core of the deployment process is executed by the `cdk deploy` command within the workflow. Because our CDK code wraps the application's Helm chart logic, this single command handles both infrastructure provisioning (such as the EKS cluster, Node Groups, and Load Balancers) and application deployment. The workflow calculates the absolute path to the local `recruitair` Helm chart and passes it to the CDK stack, which then renders the Kubernetes manifests client-side using the installed Helm CLI. The CDK code is synthesized into AWS CloudFormation templates, ensuring a deterministic deployment process that benefits from CloudFormation's inherent state management, rollback capabilities, and drift detection. All of this ensures a consistent, reproducible, and fully automated release cycle.

Milestone 6: Monitoring

Resource monitoring

RecruitAIr implements monitoring using Prometheus for metric collection and Grafana for visualization. The monitoring focuses on the FastAPI layer, as this is where applicants, criteria, job offers and scoring are handled.

Metrics are instrumented using the `prometheus_client` library in dedicated modules located at `recruitair/api/monitoring`, with a separate file for each domain. Each module defines counters to track the number of requests received, errors encountered and histograms to measure request latency.



Figure 4: Monitoring dashboard

For example, we track how frequently endpoints are called (`*_requests_total`), how often they fail (`*_requests_errors_total`), and the distribution of processing times (`*_requests_time_seconds`). These metrics make it possible to detect anomalies early, diagnose bottlenecks, monitor workload patterns and observe domain information such as CV length, job description size, or score distributions. Each module also includes domain-specific metrics (e.g., score distributions, CV length, job text length), which help detect data quality issues, anomalies, and behavioural drifts.

Metrics are exposed at <http://dispatcher-api:8000/metrics> in Prometheus format. Prometheus scrapes this endpoint periodically (every 10 seconds) and stores the values as time-series data.

The workers can also expose metrics when the following flags are enabled:

- `RECRUITAIR_EXPOSE_METRICS=true`.
- `RECRUITAIR_METRICS_SERVER_PORT=8000`

Prometheus runs as a Docker service, using the configuration file `assets/prometheus.yml` to specify scraping targets. Grafana is deployed as a separate container, and it is automatically configured using provisioning files stored in `assets/grafana`. It connects to Prometheus as a datasource and visualizes the collected metrics through curated dashboards.

Cycle: Feedback loops and retraining

The RecruitAIR pipeline is designed as a closed-loop system that naturally supports continuous improvement through monitoring, feedback, and controlled retraining. The operational cycle starts when recruiters upload job offers and applicant CVs through the Dispatcher API. Job descriptions are processed by the Criteria Extractor, which generates

structured criteria and suggested importance weights, while applicant CVs are evaluated criterion-by-criterion by the Criteria Evaluator. Throughout this process, the system persistently stores inputs, outputs, scores, model versions, and timestamps in the PostgreSQL database, providing a complete and auditable history of decisions. Monitoring components (Prometheus and Grafana) track both infrastructure metrics (latency, CPU/GPU usage) and model-related signals (score distributions, drift indicators, error rates). In addition, the system explicitly supports human-in-the-loop feedback: recruiters can manually adjust criteria descriptions, importance weights, or override evaluation scores when they disagree with the model's output. These interventions represent high-quality supervision signals that are critical for improving model behavior over time.

This feedback can be systematically reused to automate retraining pipelines. Periodically (or when drift thresholds are exceeded), updated datasets can be created from accumulated real-world data: refined criteria definitions for the extractor and corrected scores for the evaluator. These datasets feed back into the existing training and prompt-engineering pipelines already tracked in MLflow, ensuring reproducibility and lineage. New prompt versions or fine-tuned model versions are evaluated against benchmark datasets and recent production data, logged with performance and efficiency metrics, and only promoted to production after passing predefined quality gates. Thanks to model and prompt registries, retraining does not disrupt production: rollouts, A/B testing, and rollbacks are fully supported. In this way, RecruitAIr achieves a robust MLOps cycle where monitoring, human feedback, and automated retraining are tightly integrated, enabling continuous yet controlled model evolution.

Self-evaluation of the project

One of the main challenges of the project was managing the complexity of a full end-to-end MLOps system while keeping the architecture modular and understandable. Integrating multiple technologies (MLflow, FastAPI, Docker, Kubernetes, CI/CD pipelines, and monitoring tools) required careful coordination and strong conventions across repositories. Debugging distributed components (model loading from remote MLflow artifacts, inter-service communication, and Kubernetes readiness checks) proved especially demanding, but it significantly strengthened our understanding of real-world production constraints. Another challenge was balancing automation with our core design principle of keeping humans in the loop; resisting the temptation to over-automate helped us maintain transparency and interpretability, but required additional design effort. At the same time, these challenges represented major learning opportunities, particularly in infrastructure-as-code, model lifecycle management, and production-grade ML observability.

From this project, we learned how theoretical MLOps concepts translate into practical engineering trade-offs. Concepts from multiple courses were directly applied: software architecture and design patterns (microservices, workers, API gateways), cloud computing (AWS, Kubernetes, IaC with CDK), data engineering (DVC, data validation), and machine learning theory (evaluation metrics, transfer learning, bias awareness). If we were to do things differently, we would invest earlier in defining monitoring and retraining triggers, as

well as in generating synthetic but realistic user feedback data to stress-test the feedback loop sooner. Overall, the project provided a holistic view of how modern ML systems are built, deployed, monitored, and improved in practice, bridging the gap between experimentation and production-ready AI systems.